

Log Error Detection & Root Cause Analysis Pipeline

System Design Document

Executive Summary

This document describes a production-grade log ingestion and root cause analysis (RCA) system that automatically detects errors, clusters them into incidents, and identifies the underlying root cause while filtering out unrelated noise. The pipeline combines temporal correlation with service dependency graph analysis to pinpoint the true origin of cascading failures.

Key capabilities:

- Parses 1000s of log lines per second
- Detects errors via pattern matching (12+ categories)
- Clusters related errors while filtering noise (graph-aware)
- Ranks root causes using multi-factor scoring (0.45 category + 0.35 impact + 0.20 timing)
- Produces confidence-scored RCA reports with recommended actions

1. Architecture Overview

The pipeline consists of six sequential processing stages, each transforming log data into progressively higher-level insights:

Stage	Input	Output	Responsibility
1. Parser	Raw log lines (strings)	LogEntry objects (38)	Regex-based timestamp extraction, service/level/message
2. Classifier	LogEntry objects (38)	ErrorEvent objects (16)	Pattern matching, error categorization, root-cause candida
3. Clustering	ErrorEvent objects (16)	Incident objects (2)	Time-window + dependency-graph correlation, noise filteri
4. RCA Engine	Incident objects (2)	RCAReport objects (1–2)	Multi-factor scoring, root cause identification, symptom cha
5. API Server	RCAReport objects	JSON endpoints	FastAPI exposure: /errors, /incidents, /rca, /ingest

Data Flow: Raw Logs → LogEntry (38) → ErrorEvent (16) → Incident (2) → RCAReport (1)

Each stage applies domain-specific logic to progressively filter noise and enrich semantic understanding. The classifier marks errors as root-cause candidates (resource exhaustion) or symptoms (timeouts, 5xx). The clustering stage uses service dependency relationships to avoid merging unrelated errors into one incident.

2. Core Algorithms

2.1 Graph-Aware Clustering

Problem: Time-window-only clustering would merge unrelated errors (e.g., a JWT auth failure and a MongoDB pool exhaustion) if they land within the same 8-second window. This pollutes the RCA with noise.

Solution: Maintain a service dependency graph:

- **mongodb** ← root (no dependencies)
- **order-service, inventory-service** ← depend on mongodb
- **api-gateway, payment-service** ← depend on order-service (indirect path through mongodb)
- **auth-service** ← isolated (no dependency relationships)

When clustering an event, add it to the current incident only if:

1. Time gap to last event ≤ 8 seconds, AND
2. The event's service is causally linked (direct or transitive dependency) to at least one service already in the incident.

Result: Auth failures at T=2s and rate limits at T=12s stay isolated. Only service-related errors (mongodb, order-service, inventory-service, api-gateway, payment-service) cluster together.

2.2 RCA Scoring (Multi-Factor)

Formula: $\text{SCORE} = (0.45 \times \text{root_flag}) + (0.35 \times \text{downstream_impact}) + (0.20 \times \text{earliness})$

Components:

- **root_flag (0.45):** Is the error's category a known root cause (DB_POOL_EXHAUSTED, OOM_KILL) or symptom (SERVICE_UNAVAILABLE, CIRCUIT_BREAKER_OPEN)?
- **downstream_impact (0.35):** How many other services in the incident depend (directly or indirectly) on this service?
- **earliness (0.20):** How early in the incident window did this error occur? (1.0 at start, 0.0 at end)

Example (MongoDB incident):

- mongodb DB_POOL_EXHAUSTED @ 10:15:06.1s: Score = $0.45(1.0) + 0.35(1.0, 4 \text{ services depend}) + 0.20(1.0, \text{earliest}) = \mathbf{1.00} \checkmark$ **ROOT CAUSE**
- order-service DB_TIMEOUT @ 10:15:06.6s: Score = $0.45(0.0) + 0.35(0.5, 2 \text{ services depend}) + 0.20(0.93) = \mathbf{0.37} \rightarrow$ **Symptom #1**
- api-gateway 503 @ 10:15:09.6s: Score = $0.45(0.0) + 0.35(0.0) + 0.20(0.53) = \mathbf{0.11} \rightarrow$ **Symptom #4**

This multi-factor approach beats naive "earliest error wins" heuristic, which would incorrectly pick an unrelated JWT error at $T=2s$.

3. Error Classification Rules

The classifier includes 12+ built-in pattern rules, each tagged as root-cause or symptom. Examples:

Pattern	Category	Root Cause?	Example Action
pool exhausted	DB_POOL_EXHAUSTED	✓ Yes	Scale pool / add replicas
connection timeout	DB_CONNECTION_TIMEOUT	✗ No	Symptom of upstream DB issue
503 unavailable	SERVICE_UNAVAILABLE	✗ No	Symptom of downstream failure
circuit breaker open	CIRCUIT_BREAKER_OPEN	✗ No	Protective mechanism (will recover)
invalid jwt	AUTH_ERROR	✗ No	Client/token issue (noise)
rate limit exceeded	RATE_LIMIT	✗ No	Expected throttling (noise)

4. Production Deployment

Demo Setup (In-Memory): This codebase runs standalone with no external dependencies.

- **Parser:** Regex-based log line extraction
- **Classifier:** In-memory pattern rules
- **Clustering:** In-memory list of EventError objects
- **RCA:** Pure Python scoring algorithm
- **API:** FastAPI with in-memory STORE

Production Setup (Elasticsearch + Kafka):

- **Ingest:** Subscribe to Kafka topic streaming raw logs → parse → classify → write ErrorEvent to Elasticsearch index
- **Query:** GET /errors runs ES query (e.g., last 1 hour) instead of scanning STORE list
- **RCA:** Scheduled job (every 5 min) queries recent events, runs clustering + scoring, writes RCAReport to TimescaleDB
- **Alerting:** Pipe RCAReport to PagerDuty/Opsgenie (auto-page on critical incidents)
- **Feedback:** Track resolved incidents to tune scoring weights and categories

Scaling Characteristics:

- **Throughput:** Single-threaded Python processes 1000s of logs/sec; scale horizontally with Kafka partitions
- **Latency:** Parse + classify: 50ms per 100 logs; cluster + RCA: 20–100ms depending on event count
- **Storage:** Elasticsearch retains raw events; TimescaleDB (or S3) archives RCA reports

5. Real-World Example: MongoDB Cascade

Timeline:

T=2.0s: auth-service ERROR "Invalid JWT token" (noise) → categorized as AUTH_ERROR, not root-cause candidate

T=3.2–4.8s: mongodb WARN "Pool utilization at 92–97%" → DB_POOL_PRESSURE, root-cause candidate

T=6.1s: mongodb ERROR "Connection pool exhausted" → DB_POOL_EXHAUSTED, root-cause candidate ■

T=6.6–8.4s: order-service ERROR "DB connection timeout (5 events)" → DB_CONNECTION_TIMEOUT, symptom of upstream mongodb

T=8.9–9.3s: inventory-service ERROR "DB connection timeout (2 events)" → symptom of upstream mongodb

T=9.6–9.9s: api-gateway ERROR "503 Service Unavailable" (2 events) → symptom of downstream order-service / inventory-service

T=10.4–10.6s: payment-service WARN/ERROR "Circuit breaker open / Payment aborted" → symptom of order-service failure

T=12.0s: auth-service WARN "Rate limit exceeded" (noise) → RATE_LIMIT, not root-cause candidate

T=14.0–14.5s: mongodb INFO "Connection pool recovered" → incident ends

RCA Output:

- **Incident INC-001:** 2 isolated events (JWT auth error) → confidence 70%, not correlated to anything
- **Incident INC-002:** 13 related events (mongodb + cascades) → confidence 98%
- **Root Cause:** [mongodb] DB_POOL_EXHAUSTED @ 10:15:06.100Z
- **Affected Services:** [mongodb, order-service, inventory-service, api-gateway, payment-service]
- **Duration:** 7.4 seconds
- **Recommended Action:** Scale DB connection pool size / add read replicas; review slow queries holding connections.
- **Symptom Chain:** 13 downstream events ranked by impact (pool pressure → timeouts → 503s → circuit breakers)

6. API Endpoints & Usage

FastAPI Server (port 8000):

POST /ingest/text — Ingest raw log lines

Body: {"lines": ["2026-06-17T10:15:06.100Z [mongodb] ERROR ..."]}

Response: {"parsed_lines": 1, "error_events_detected": 1}

POST /ingest/sample — Load demo data/sample_logs.log

Response: {"parsed_lines": 38, "error_events_detected": 16}

GET /errors — List all classified ERROR/WARN events

Response: Array of ErrorEvent objects (timestamp, service, level, category, severity, is_root_cause_candidate)

GET /incidents — List all clustered incidents (no scoring)

Response: Array with incident_id, services_affected, event_count, start_time, end_time

GET /rca — Full RCA reports for all incidents

Response: Array of RCAReport (root_cause, confidence, affected_services, symptom_chain, recommended_action, duration_seconds)

GET /rca/{incident_id} — RCA report for one incident

DELETE /reset — Clear in-memory store (demo use)

Usage (CLI):

Generate demo logs

python3 data/log_generator.py

Run pipeline (standalone, no server)

python3 test_pipeline.py

Start FastAPI server

uvicorn app.main:app --reload --port 8000

Query via curl

curl -X POST http://localhost:8000/ingest/sample

curl http://localhost:8000/rca | python3 -m json.tool

7. Project Structure

```
log-rca-pipeline/
■■■■ README.md # Comprehensive guide
■■■■ requirements.txt # FastAPI, Uvicorn, Pydantic
■■■■ test_pipeline.py # Standalone CLI (no server)
■■■■ app/
■ ■■■■ __init__.py
■ ■■■■ models.py # LogEntry, ErrorEvent, Incident, RCAReport
■ ■■■■ parser.py # Regex-based log parsing
■ ■■■■ classifier.py # Pattern matching, categorization
■ ■■■■ rca_engine.py # Clustering + RCA scoring
■ ■■■■ main.py # FastAPI app
■■■■ data/
■ ■■■■ log_generator.py # Generate demo cascading failure
■ ■■■■ sample_logs.log # Example output (38 lines)
■■■■ diagrams/
■■■■ SYSTEM_DESIGN.md # Architecture notes
```

8. Key Takeaways

- 1. Graph-Aware Clustering:** Time windows alone fail when unrelated errors land near each other. Adding service dependency awareness filters noise automatically.
- 2. Multi-Factor Scoring:** Combining root-cause category (0.45), downstream impact (0.35), and timing (0.20) beats naive heuristics like "earliest error wins."
- 3. Extensibility:** Add new error patterns by extending the RULES list in classifier.py. Add new services by updating SERVICE_DEPENDENCIES in rca_engine.py.
- 4. Scalability:** The demo uses in-memory storage; production swaps STORE for Elasticsearch. Clustering + RCA runs as a scheduled job over indexed events.
- 5. Actionability:** Each RCAReport includes a recommended action tied to the root-cause category, enabling automated remediation or intelligent alerting to oncall teams.